MACQUARIE University

COMP8221 Advanced Machine Learning

Heterogeneous Graphs

Zitai Qiu, School of Computing, zitai.qiu@mq.edu.au

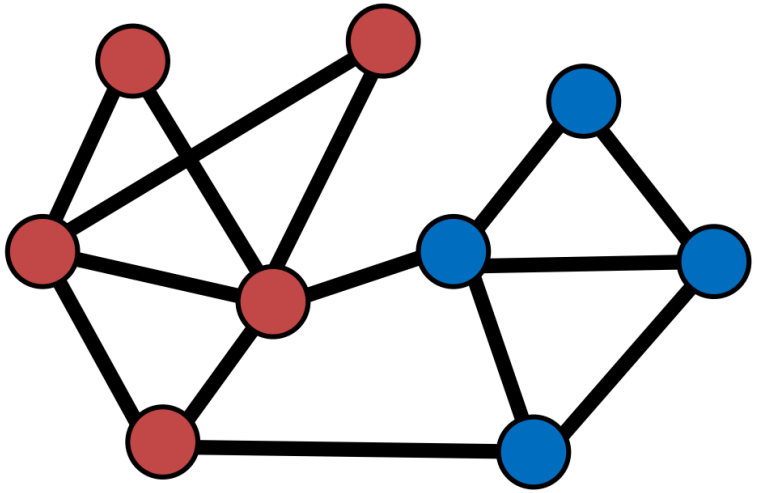


Outline

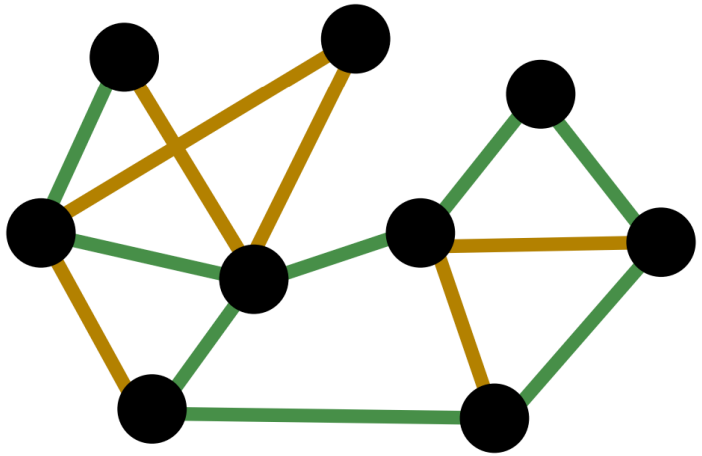
- Heterogeneous Graphs
  - Relational GCNs
  - Design space for heterogeneous GNNs

Motivation

- 2 types of nodes:
  - Paper nodes
  - Author nodes



- 2 types of edges:
  - Like edge
  - Cite edge



Motivation

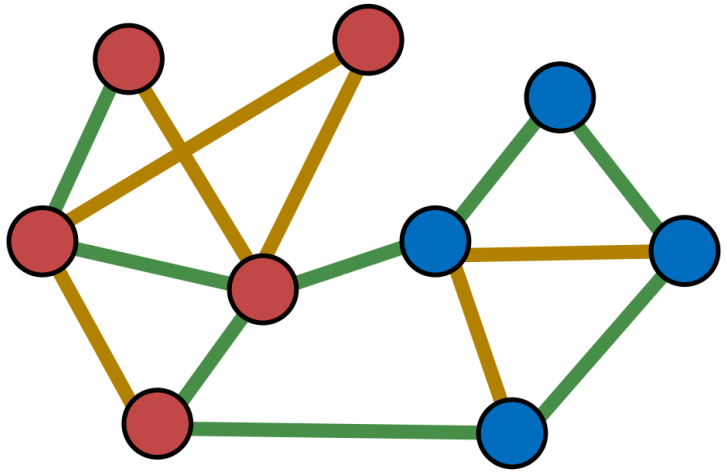
- A graph could have multiple types of nodes and edges!
- 2 types of nodes + 2 types of edges
- 8 possible relation types!

(Paper, Cite, Paper)

(Paper, Like, Paper)

(Paper, Cite, Author)

(Paper, Like, Author)



(Author, Cite, Author)

(Author, Like, Author)

(Author, Cite, Paper)

(Author, Like, Paper)

Relation types

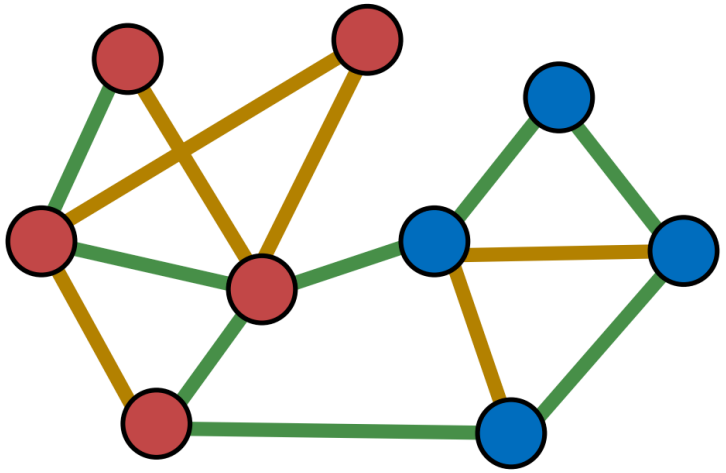
- Relation types: (node_start, edge, node_end)
- We use relation type to describe an edge
- Relation type better captures the interaction between nodes and edges

(Paper, Cite, Paper)

(Paper, Like, Paper)

(Paper, Cite, Author)

(Paper, Like, Author)



(Author, Cite, Author)

(Author, Like, Author)

(Author, Cite, Paper)

(Author, Like, Paper)

Heterogeneous graph

MACQUARIE University

- A heterogeneous graph is defined as

$$G = (V, E, \tau, \phi)$$

- V is the vertex set
- E is the edge set

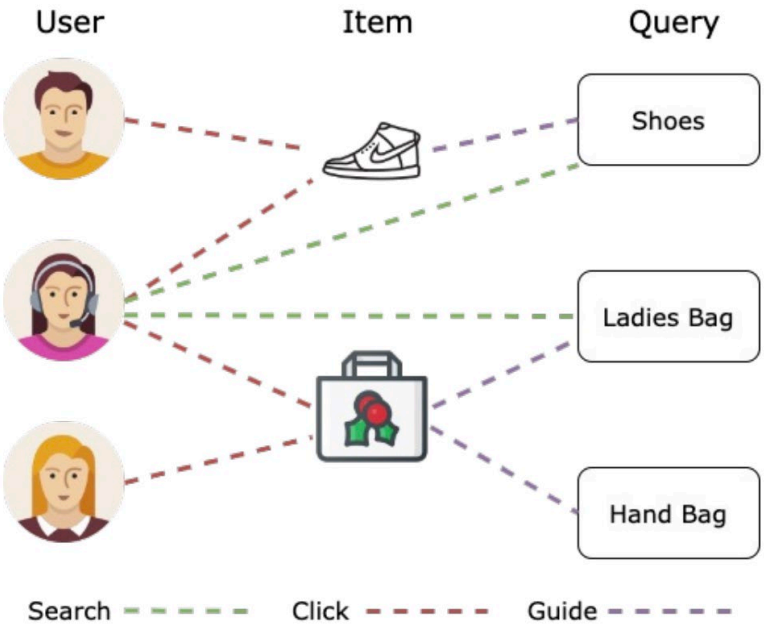
- For each node $v \in V$, node type is $\tau(v)$
- For each edge $(u, v) \in E$, edge type is $\phi(u, v)$

- Relation type:

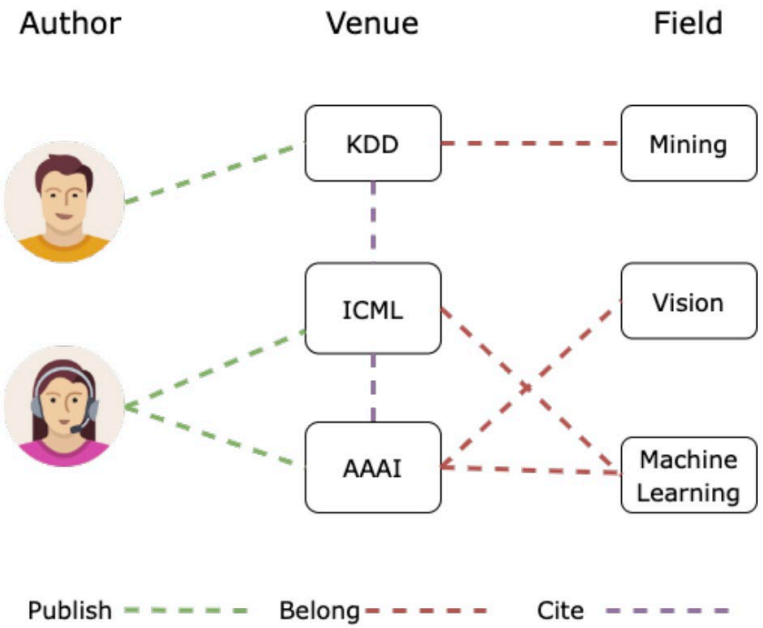
$$r(u, v) = (\tau(u), \phi(u, v), \tau(v))$$

Heterogeneous graph

- Example: E-Commerce Graph
- Node type
- Edge type



- Example: Academic Graph
- Node type
- Edge type
- Microsoft Academic Graph



Type vs Features

MACQUARIE University

- We can also treat types of nodes and edges as features
- Example:

  Add a one-hot indicator for nodes and edges

  Append feature [1, 0] to each “author node”;

  Append feature [0, 1] to each “paper node”

  Similarly, we can assign edge features to edges with different types

- Then, a heterogeneous graph reduces to a standard(homogeneous) graph.

- **When do we need a heterogeneous graph?**

Type vs Features

MACQUARIE University

- When do we need a heterogeneous graph?

  Case 1: Different node/edge types have **different shapes of features**

  A “author node” has 4-dim feature

  A “paper node” has 5-dim feature

  Case 2: We know different relation types represent **different types of interactions**

  (English, translate, French) and (English, translate, Chinese) require different models

Type vs Features

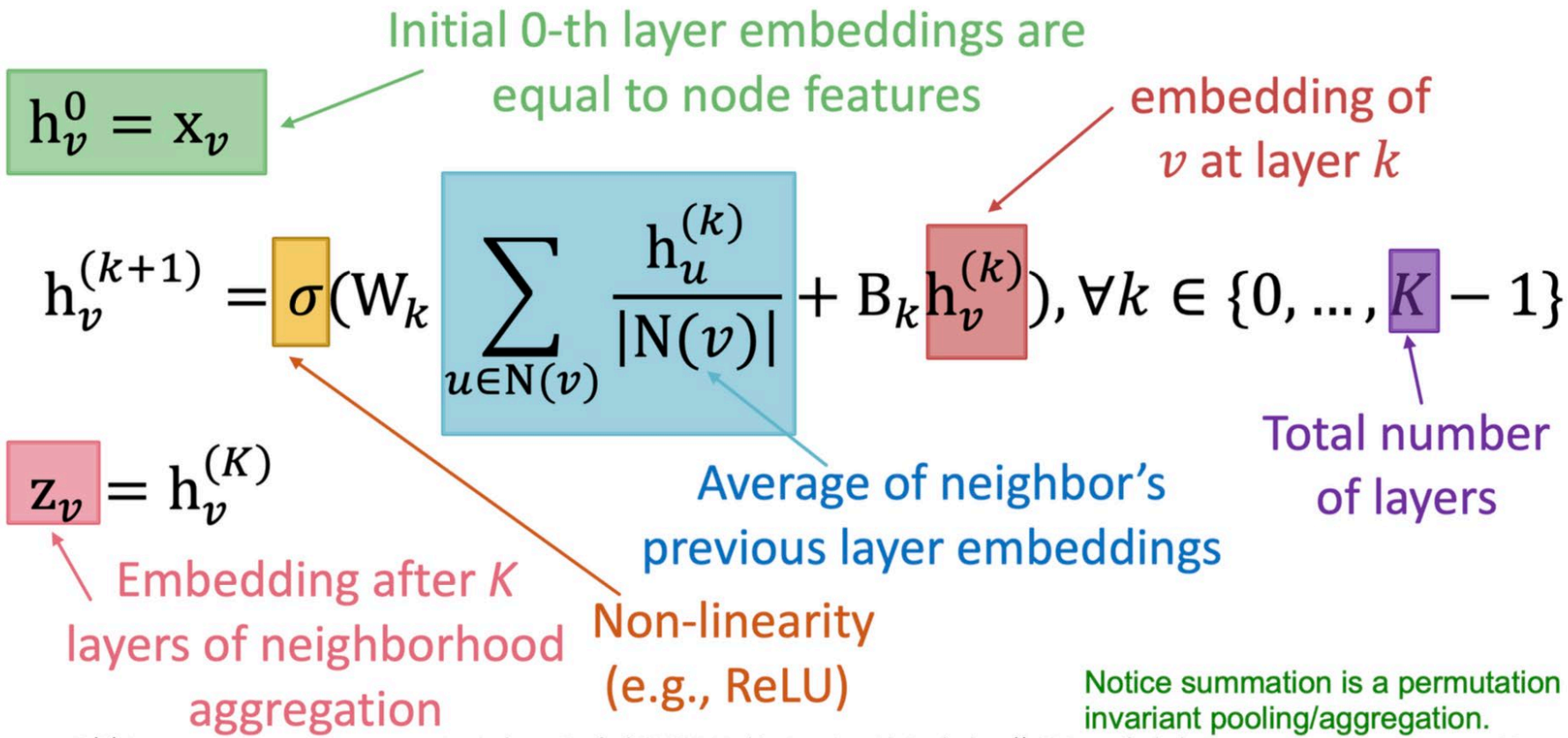


MACQUARIE University

- Heterogeneous graph is a more expressive graph representation
  - Captures different types of interactions between entities

- But it also comes with costs
  - More expensive (computation, storage)
  - More complex implementation

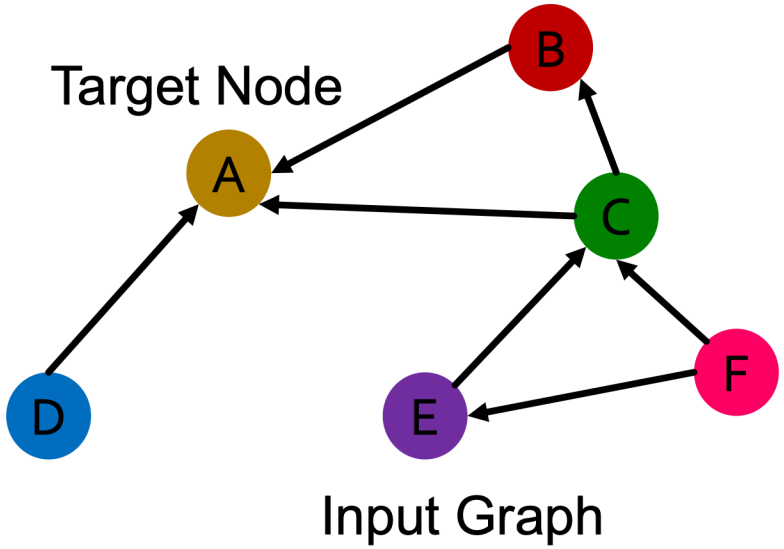
Recap: Classical GNN -- GCN



Relational GCN

- Extend GCN to handle heterogeneous graphs with multiple edge/relation types
- We start with a directed graph with **one** relation

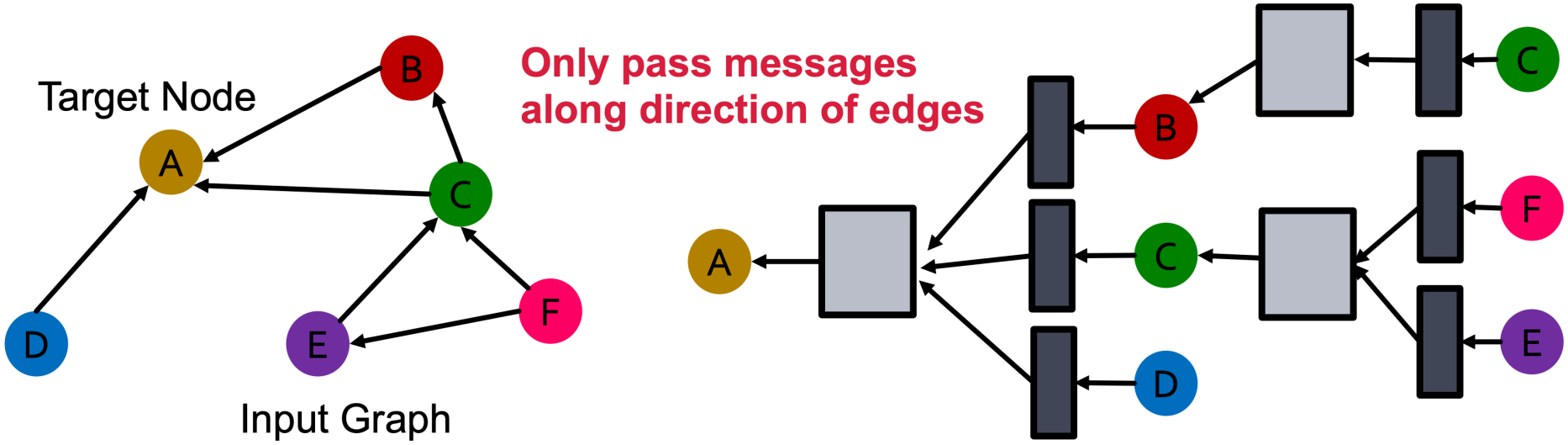
How do we run GCN and update the representation of the **target node A** on this graph?



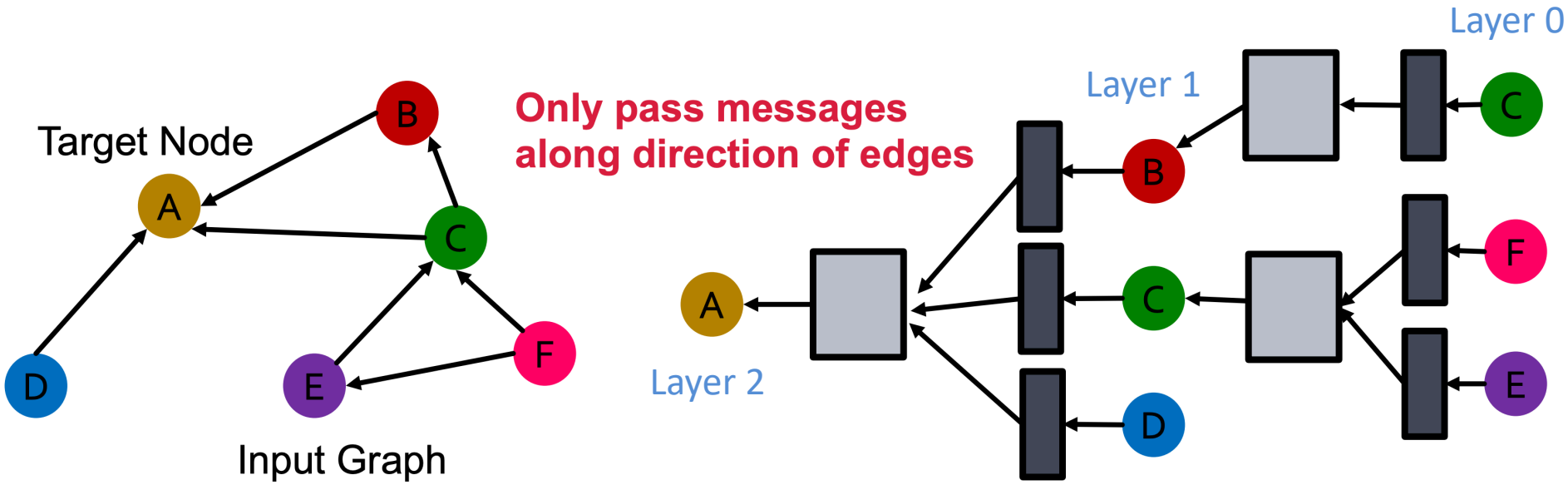
Relational GCN

- Extend GCN to handle heterogeneous graphs with multiple edge/relation types
- We start with a directed graph with **one** relation

How do we run GCN and update the representation of the **target node A** on this graph?

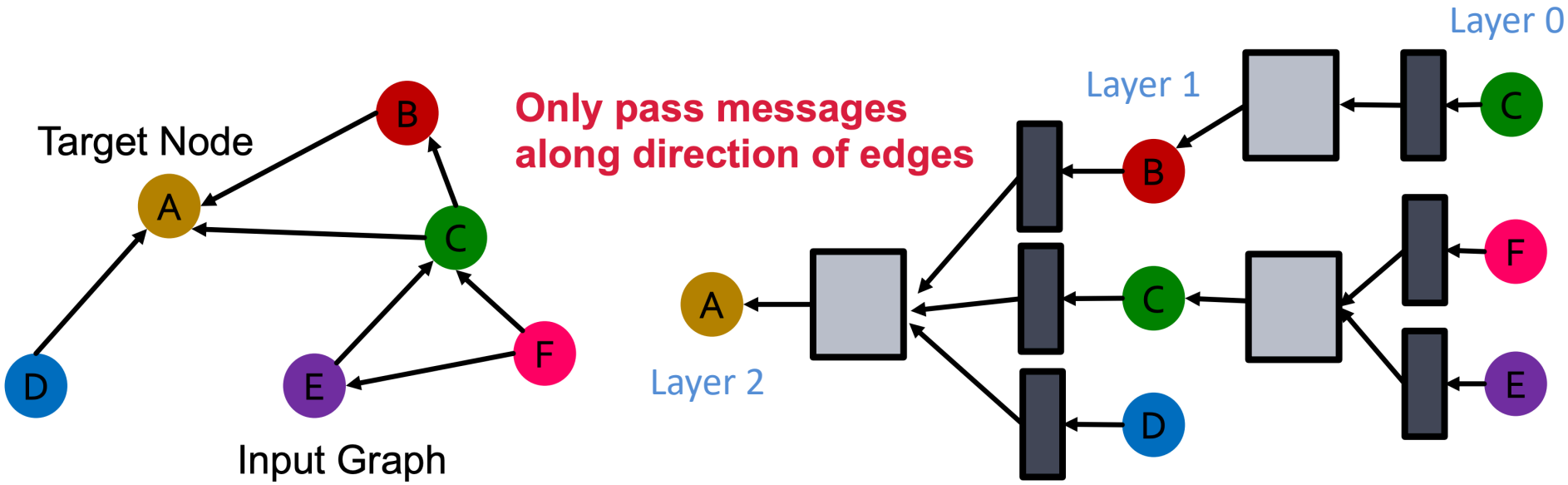


Relational GCN



- Assume initial node features are denoted as $h_A^0, h_B^0, h_C^0, h_D^0, h_E^0, h_F^0$
- Use W^l, B^l as weights for each layer l
- What is $\boldsymbol{h_A^1, h_B^1, h_C^1, h_D^1}$ and $\boldsymbol{h_A^2}$?

Relational GCN



$$h_A^1 = \sigma\left(\frac{W^1 h_B^0 + W^1 h_C^0 + W^1 h_D^0}{3} + B^1 h_A^0\right)$$

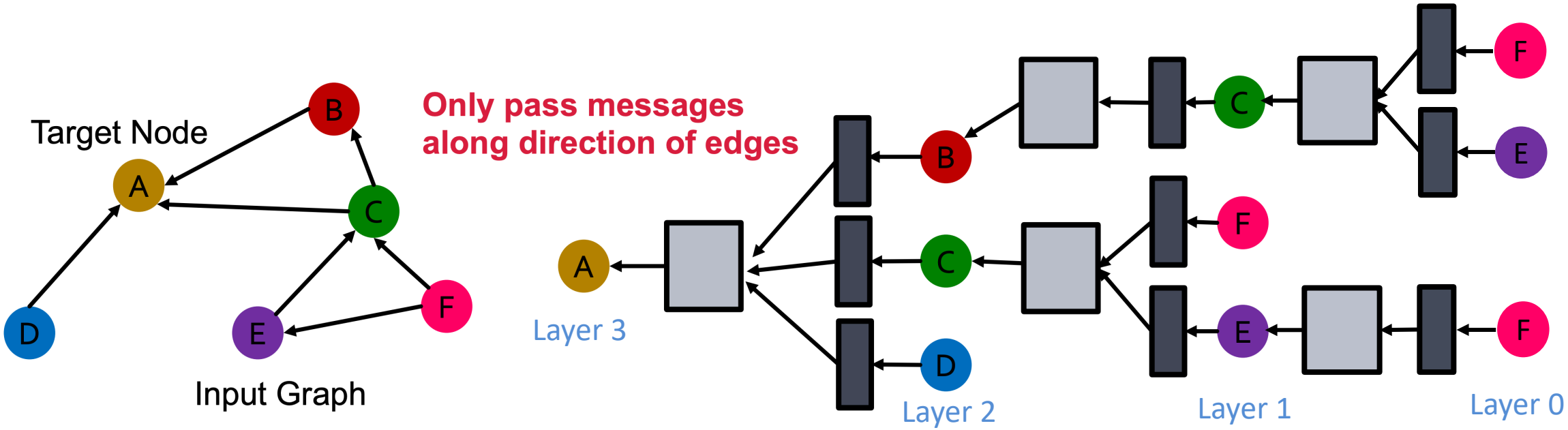
$$h_B^1 = \sigma\left(W^1 h_C^0 + B^1 h_B^0\right)$$

$$h_C^1 = \sigma\left(\frac{W^1 h_F^0 + W^1 h_E^0}{2} + B^1 h_C^0\right)$$

$$h_D^1 = \sigma(B^1 h_D^0)$$

$$h_A^2 = \sigma\left(\frac{W^2 h_B^1 + W^2 h_C^1 + W^2 h_D^1}{3} + B^2 h_A^1\right)$$

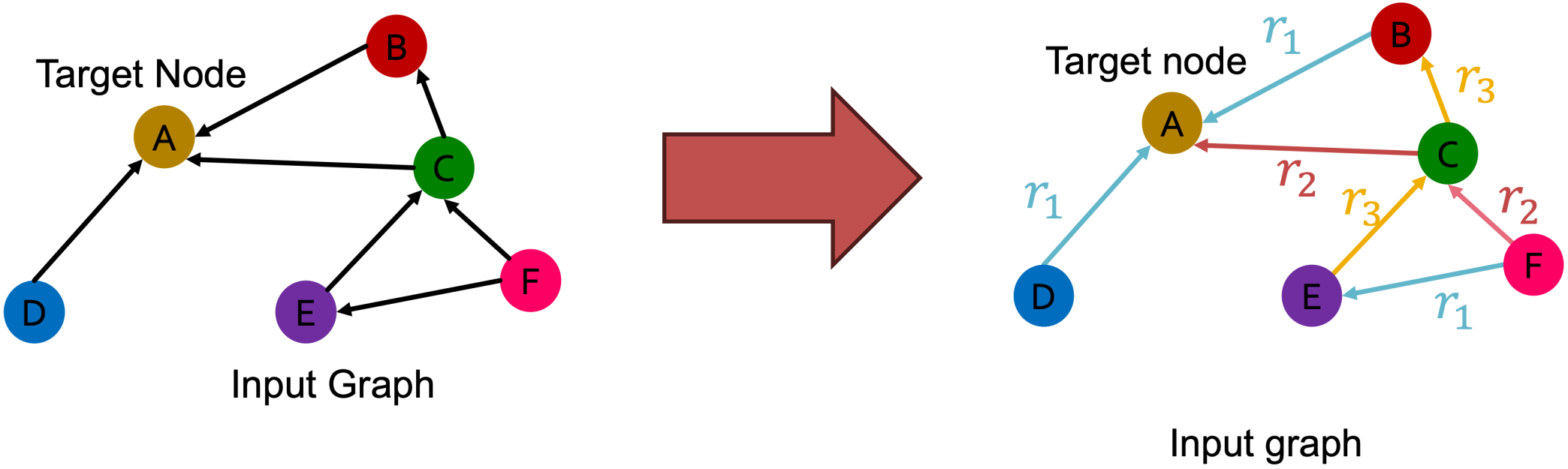
Relational GCN



- Assume initial node features are denoted as $h_A^0, h_B^0, h_C^0, h_D^0, h_E^0, h_F^0$
- Use W^l, B^l as weights for each layer l
- What is $\boldsymbol{h_A^3}$?

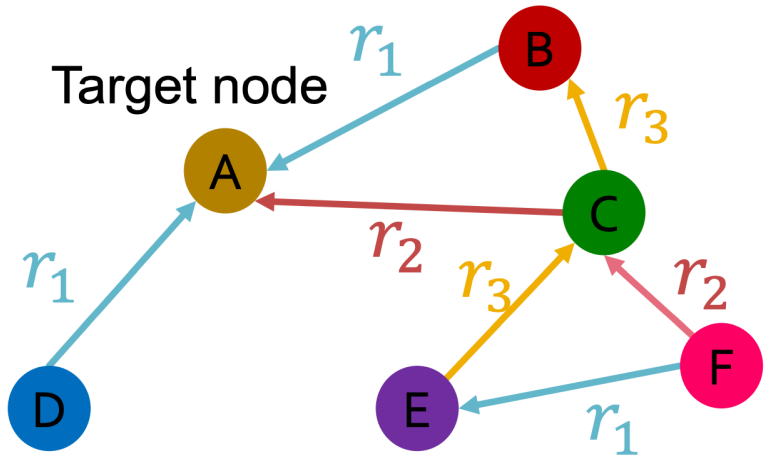
Relational GCN

- What if the graph has **multiple relation types**?

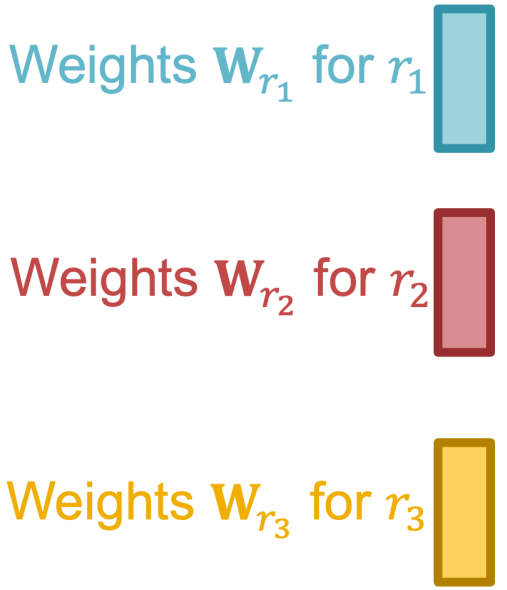


Relational GCN

- What if the graph has **multiple relation types**?
- Use different neural network weights for different relation types.

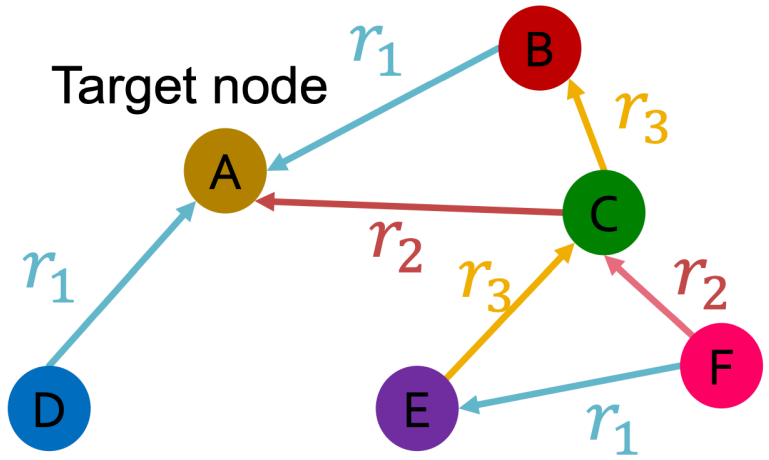


Input graph

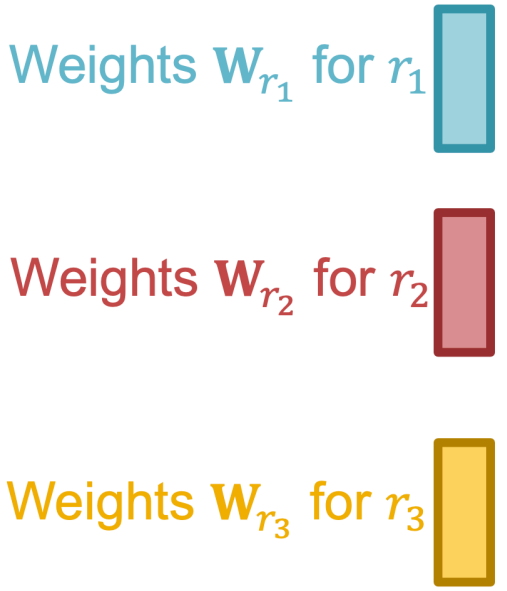


Relational GCN

- Can you draw the computing node for the target node A?

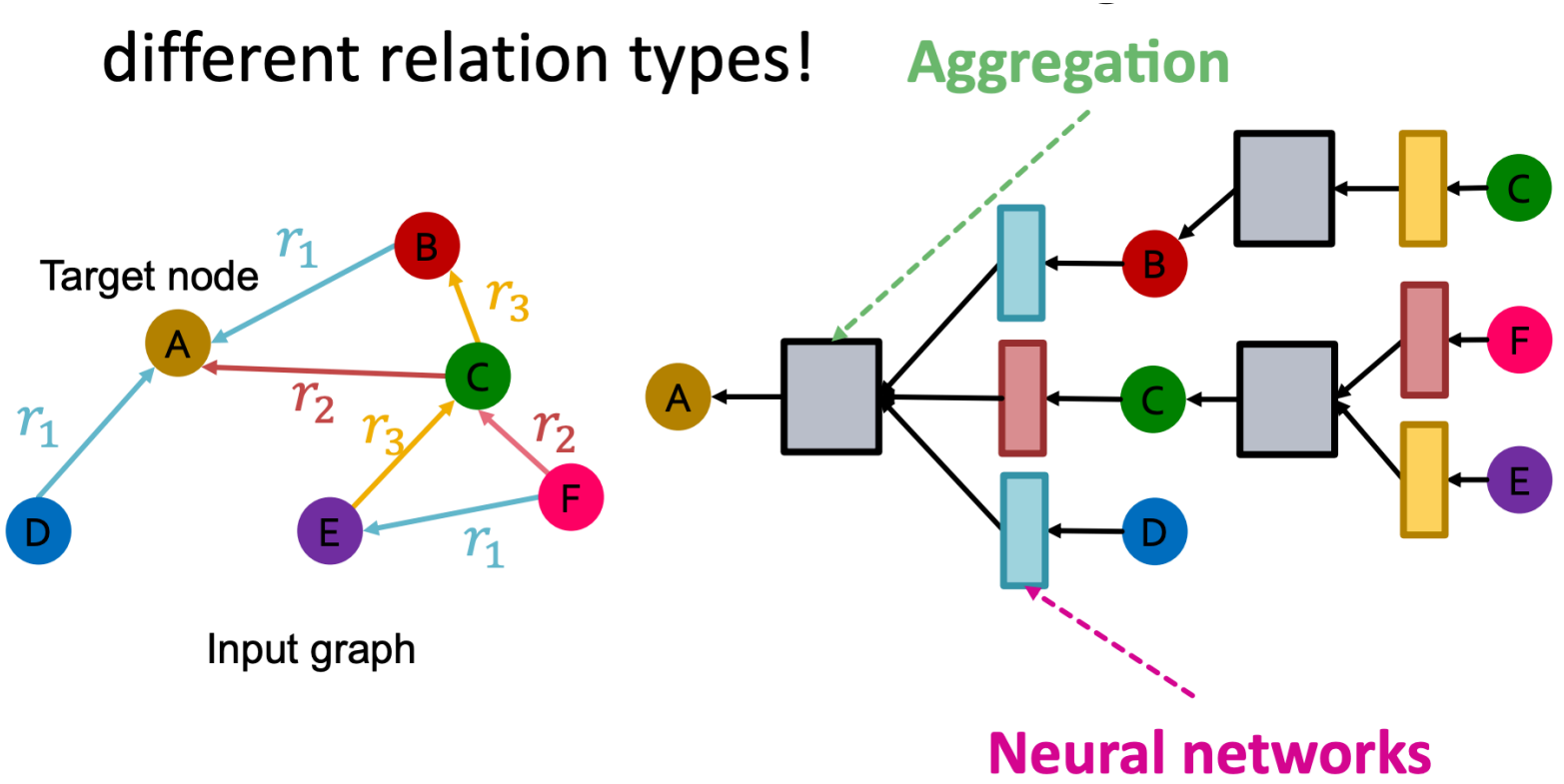


Input graph



Relational GCN

- Can you draw the computing node for the target node A?



Relational GCN

- Relational GCN (RGCN):

$$h_v^{(l+1)} = \sigma\left(\sum_{r \in R} \sum_{u \in N_v^r} \frac{1}{|N_v^r|} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)}\right)$$

For each relation r

For each neighbor u in this relation r

Number of neighbors for this relation r

Node feature of layer l for neighbor u

Node feature of layer l for target v

Relational GCN

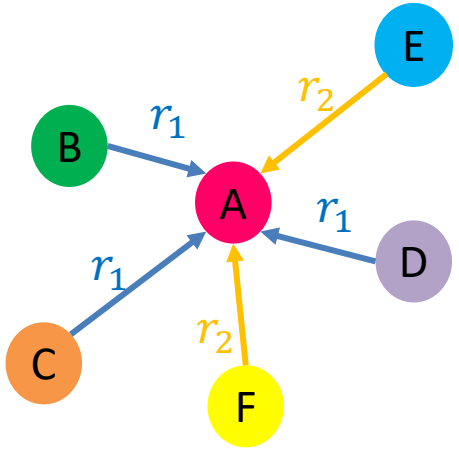
- Relational GCN (RGCN):

$$h_v^{(l+1)} = \sigma\left(\sum_{r \in R} \sum_{u \in N_v^r} \frac{1}{|N_v^r|} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)}\right)$$

Consider this simple heterogenous graph with 2 types of relations.

Assume initial node features are denoted as $h_A^0, h_B^0, h_C^0, h_D^0, h_E^0, h_F^0$

What is h_A^1?



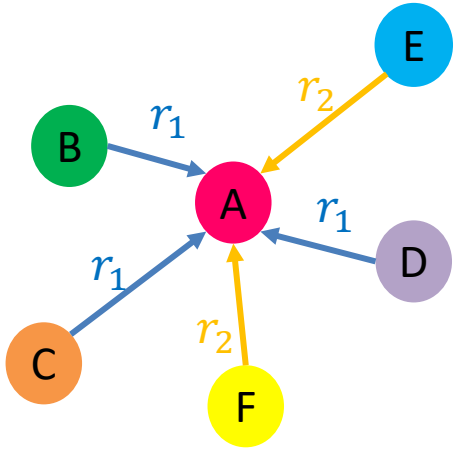
Relational GCN

- Relational GCN (RGCN):

$$h_v^{(l+1)} = \sigma\left(\sum_{r \in R} \sum_{u \in N_v^r} \frac{1}{|N_v^r|} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)}\right)$$

Consider this simple heterogenous graph with 2 types of relations.

Assume initial node features are denoted as $h_A^0, h_B^0, h_C^0, h_D^0, h_E^0, h_F^0$



What is h_A^1?

$$h_A^1 = \sigma\left(\frac{W_{r1}^0 h_B^0 + W_{r1}^0 h_C^0 + W_{r1}^0 h_D^0}{3} + \frac{W_{r2}^0 h_E^0 + W_{r2}^0 h_F^0}{2} + W_0^0 h_A^0\right)$$

Relational GCN: Scalability

MACQUARIE University

- Each relation has L matrices: $W_r^{(1)}$, $W_r^{(2)}, W_r^{(3)}, \ldots, W_r^{(L)}$
- The size of each $W_r^{(l)}$ is $d^{(l+1)} \times d^l$, → d^l is the hidden dimension in layer l

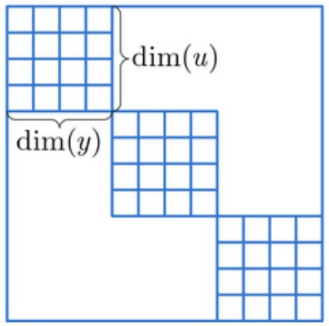
- Rapid growth of the number of parameters w.r.t number of relations
  → **Overfitting**

- Two methods to regularize the weights
  1) Block Diagonal matrices
  2) Basis/Dictionary learning

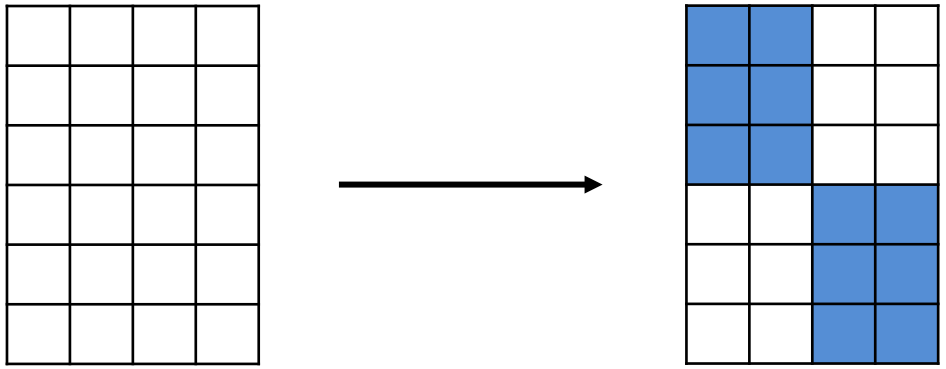
Block Diagonal Matrices

- Use block diagonal matrices for weights matrices

$$\mathbf{W}_r = \text{Block diagonal}$$



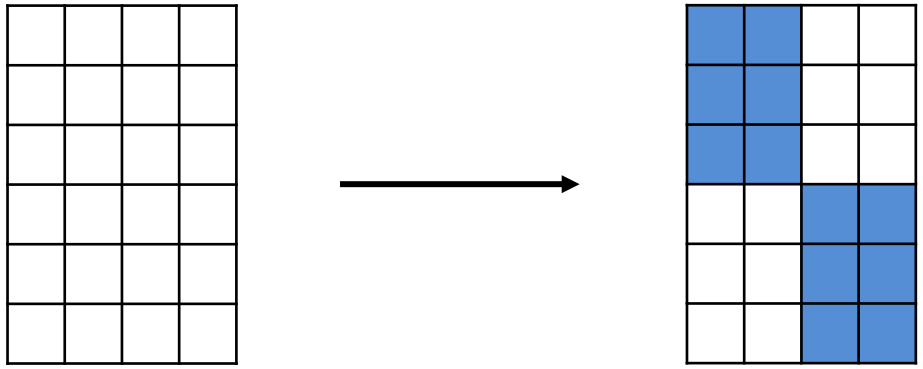
- For example, assume original W_r is 6×4 and we use **2 blocks**:



Block Diagonal Matrices

MACQUARIE University

- For example, assume original W_r is 6×4 and we use **2 blocks**:



- The number of parameters reduces from $6 \times 4 = 24$ to $2 \times \frac{6}{2} \times \frac{4}{2} = 12$
- So if we use B blocks for W_r, then num param reduces from $d^{(l+1)} \times d^l$ to $\boldsymbol{B \times \frac{d^{(l+1)}}{B} \times \frac{d^l}{B}}$
- Any issues?

Block Diagonal Matrices



MACQUARIE University

The block diagonal matrices can't mix information **between** blocks. That is, features in one block can only be transformed via their corresponding submatrix and **do not interact with features in other blocks**.

Example

Suppose you have a node feature vector of size 6:

$$x = [x_1, x_2, x_3, x_4, x_5, x_6]^T$$

Now imagine using a **block diagonal** weight matrix with 2 blocks:

$$W = \begin{bmatrix} W_1 & 0 \\ 0 & W_2 \end{bmatrix} \text{ where } W_1, W_2 \in R^{3\times 3}$$

Block Diagonal Matrices

MACQUARIE University

Example

Suppose you have a node feature vector of size 6:

$$x = [x_1, x_2, x_3, x_4, x_5, x_6]^T$$

Now imagine using a **block diagonal** weight matrix with 2 blocks:

$$W = \begin{bmatrix} W_1 & 0 \\ 0 & W_2 \end{bmatrix} \text{ where } W_1, W_2 \in R^{3\times 3}$$

That means:

x_1, x_2, x_3 will be transformed only via W_1

x_4, x_5, x_6 will be transformed only via W_2

There's **no way for** x_1 **to influence** x_4 and vice versa.

When aggregate messages from neighbouring nodes, features in dim 1–3 will only be influenced by other features in dim 1–3.→ **Cross-dimensional interaction is limited.**

Basis Learning

MACQUARIE University

- Key: Share weights across different relations
- Represent the matrix of each relation as a linear combination of basis transformations:

$$W_r = \sum_{b=1}^{B} a_r^b \cdot V_b$$

- $\{V_b\}_{b=1}^{B}$ are the basis matrices and are shared across all relations
- a_r^b is the importance weights for relation r and basis matrix V_b

Basis Learning: Example

MACQUARIE University

Imagine a heterogeneous graph with nodes of types **Author** and **Paper**, and two types of edges:

- **writes** (from Author to Paper)
- **cites** (from Paper to Paper)

- For example, let's set the number of bases to $B = 2$
- Then we have:

  For layer l, shared basis matrices: $V_1^{(l)} \in \mathbb{R}^{d^{(l+1)} \times d^l}, V_2^{(l)} \in \mathbb{R}^{d^{(l+1)} \times d^l}$

  Learnable coefficients for each relation:

  Relation "writes": $a_{writes}^1, a_{writes}^2$

  Relation "cites": a_{cites}^1, a_{cites}^2

  The relation weight matrices: $W_{writes}^{(l)} = a_{writes}^1 V_1^{(l)} + a_{writes}^2 V_2^{(l)}$

  $W_{cites}^{(l)} = a_{cites}^1 V_1^{(l)} + a_{cites}^2 V_2^{(l)}$

Basis Learning: Example

MACQUARIE University

- Typically, basis matrices are **not shared across layers**. Each layer usually has its own set of basis matrices.

- **Learnable Parameters:**

  Basis matrices and the **coefficients** are all learnable.
  They will be **updated during training** via backpropagation, optimizing for the specific graph prediction task (classification, link prediction, etc.).

- The number of parameters for each layer changes from $d^{(l+1)} \times d^{l} \times |R|$ to

$$\boldsymbol{d^{(l+1)} \times d^{l} \times B + |R| \times B = (d^{(l+1)} \times d^{l} + |R|) \times B}$$

- **What can you tell from the equation?**

Basis Learning: Example

MACQUARIE University

- The number of parameters for each layer changes from $d^{(l+1)} \times d^{l} \times |R|$ to

$$d^{(l+1)} \times d^{l} \times B + |R| \times B = (d^{(l+1)} \times d^{l} + |R|) \times B$$

- **What can you tell from the equation?**

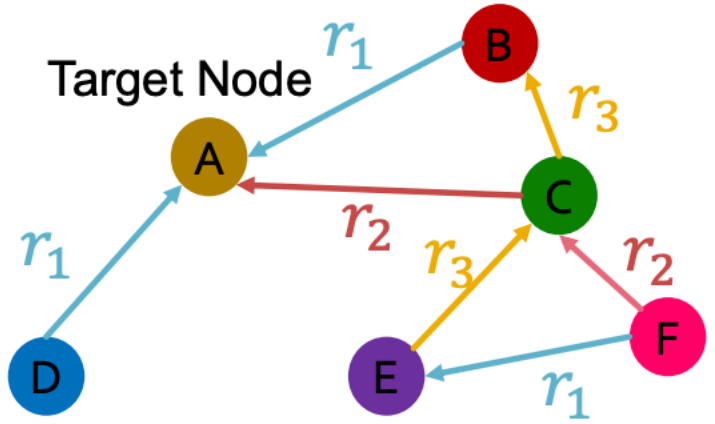
When $B \ll |R|$, Basis learning significantly reduces the number of parameters.
When $B \approx |R|$, the parameter savings diminish.

RGCN: Node Classification

- Goal: Predict the label of a given node
- RGCN uses the representation of the final layer:

  If we predict the class of node A from K classes

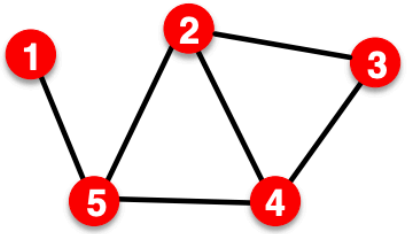
  Take the final layer (prediction head): $h_A^{(L)} \in \mathbb{R}^K$, each item in $h_A^{(L)}$ represents the probability of that class



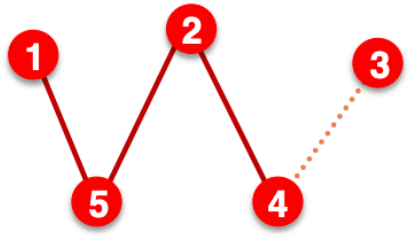
RGCN: Link Prediction

MACQUARIE University

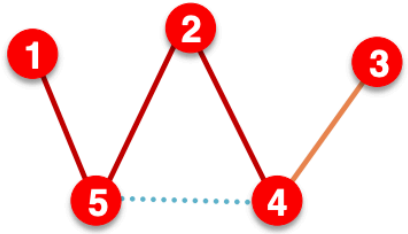
- Recap:
  - **Transductive link prediction split:**



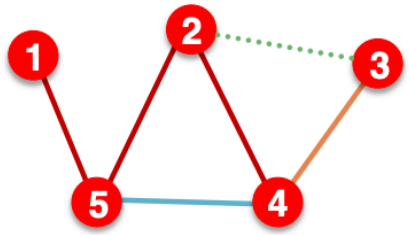
The original graph



(1) At training time:
Use **training message edges** to predict **training supervision edges**



(2) At validation time:
Use **training message edges & training supervision edges** to predict **validation edges**



(3) At test time:
Use **training message edges & training supervision edges & validation edges** to predict **test edges**

RGCN: Link Prediction

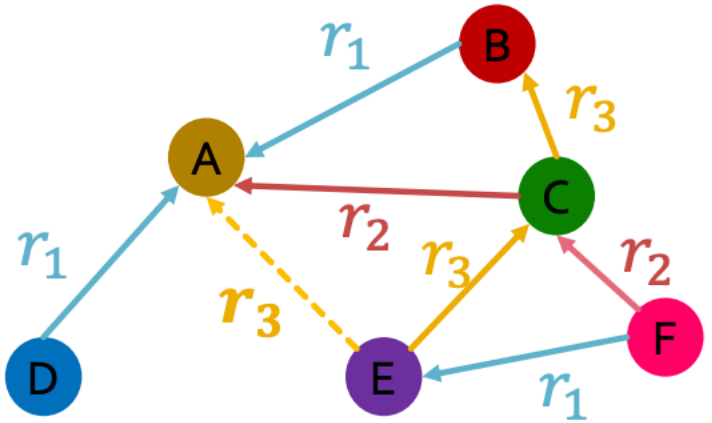
- Link prediction: Make prediction using pairs of node embeddings
- Assume (E, r_3, A) **is the training supervision edge**.
- All other edges are **training message edges** (solid lines).

- Use RGCN to predict (E, r_3, A)

Take final layer of node E and A: $h_E^{(L)},\ h_A^{(L)} \in \mathbb{R}^d$

Relation-specific function: f_r: $\mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$

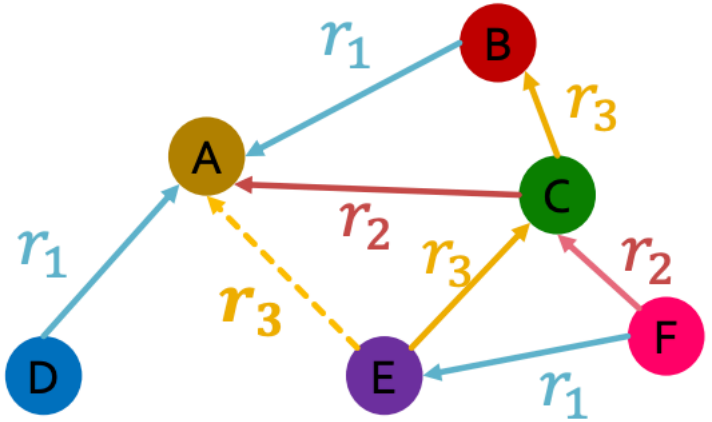
For example, $f_3(h_E^{(L)}, h_A^{(L)}) = (h_E^{(L)})^T W_3 h_A^{(L)}$



RGCN: Link Prediction

- Link prediction: Make prediction using pairs of node embeddings
- **Create negative edges** by perturbing the supervision edge (E, r_3, A) → e.g. (E, r_3, B).

Note the negative edges should NOT belong to training/supervision edge: e.g (E, r_3, C) is **NOT** a negative edge



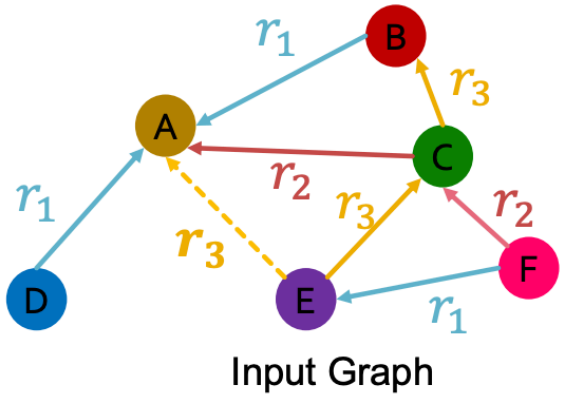
Input Graph

RGCN: Link Prediction

- Link prediction: Make prediction using pairs of node embeddings
- **Training:**
  1. Use **RGCN to score** the training supervision edge (E, r_3, A)
  2. Create a negative edge (E, r_3, B)
  3. Use RGCN model to score negative edge
  4. Optimize a standard cross entropy

     Maximize the score of training supervision edge

     Minimize the score of negative edge

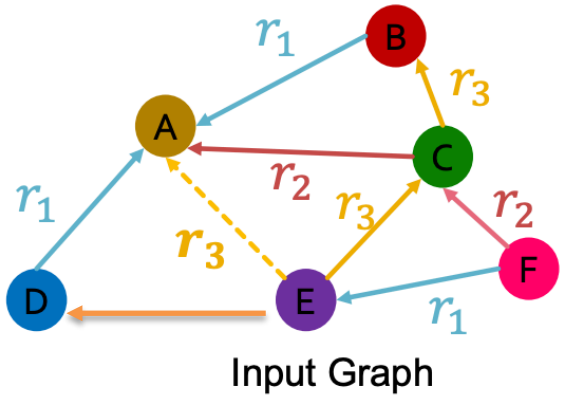


Input Graph

$$l = -log\sigma\left(f_{r_3}\left(h_E^{(L)}, h_A^{(L)}\right)\right) - log\left(1 - \sigma\left(f_{r_3}\left(h_E^{(L)}, h_B^{(L)}\right)\right)\right), \sigma \ldots \text{sigmoid function}$$

RGCN: Link Prediction

- Link prediction: Make prediction using pairs of node embeddings
- **Evaluation:**
- Let's predict validation edge (E, r_3, D)
  1. Calculate the score of (E, r_3, D)
  2. Calculate the score of all the negative edges: $\{(E, r_3, v) | v \in \{B, F\}\}$,
     Since (E, r_3, A) and (E, r_3, C) belong to training message edges & training supervision edges
  3. Obtain the ranking RK of (E, r_3, D)
  4. Calculate metrics:
     Hits@k: $1[RK \leq k]$. Higher is better
     Reciprocal Rank: $\frac{1}{RK}$. Higher is better



Input Graph

Intuition: the score of (E, r_3, D) should be higher than all other negative edges that are NOT in the training message edges and training supervision edges,

RGCN: Link Prediction

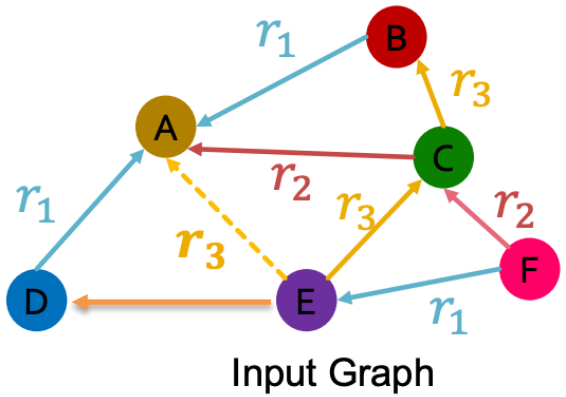
- Link prediction: Make prediction using pairs of node embeddings
- **Evaluation:**

  For example:

  The ranking: $f_{r_3}(E, r_3, D)$

  $f_{r_3}(E, r_3, B)$

  $f_{r_3}(E, r_3, F)$



Input Graph

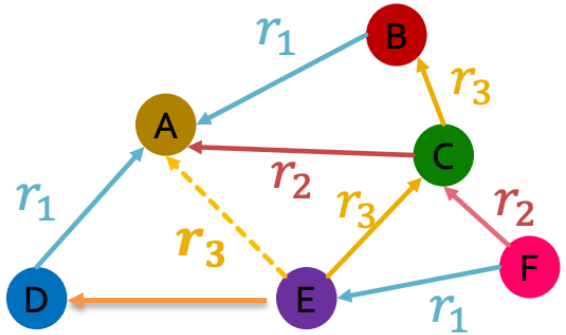
RK of (E, r_3, D): 1

1. If k = 2 (top 2 predictions), **Hits@k**: $1[RK \leq 2] \rightarrow 1$
2. Reciprocal Rank: $\frac{1}{RK} = \frac{1}{1} = 1$

RGCN: Link Prediction

- Link prediction: Make prediction using pairs of node embeddings
- **Evaluation:**

The goal is to see how well the model predicts the true edge (E, r_3, D) by comparing its score against various negative edges.



Input Graph

The **evaluation metrics** are standard for knowledge graph link prediction tasks:

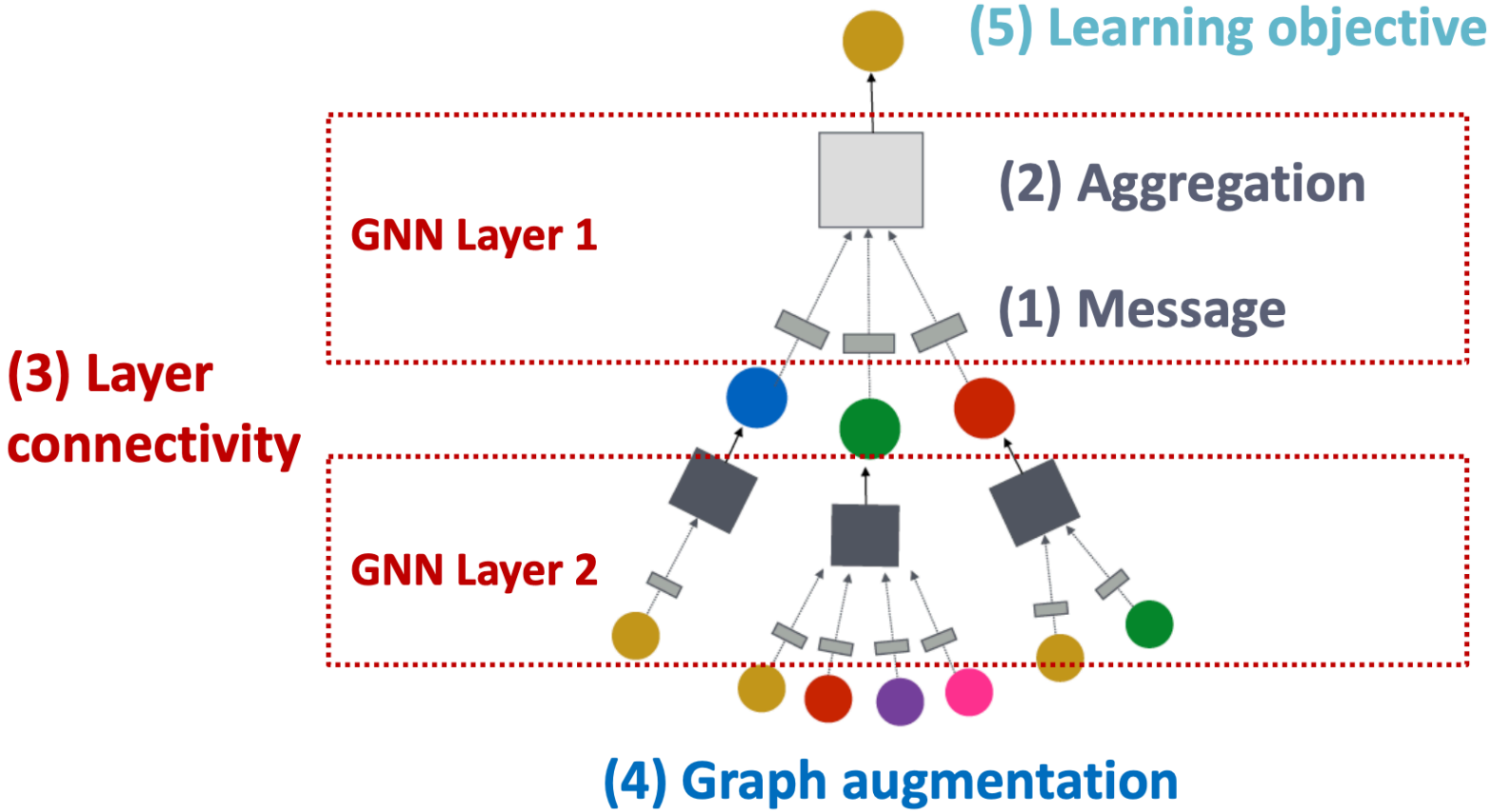
Hits@k: Measures if the true edge ranks within the top-k predictions.

Reciprocal Rank: Evaluates the ranking quality by giving higher scores to top-ranked predictions.

Recap: GNN Framework

MACQUARIE University

How do we extend the general GNN design space to heterogneous graphs?



Message Computation

- A node could receive multiple types of messages. Num of message type = Num of relation type

- Idea: Create a different message function for each relation type

$$m_u^{(l)} = MSG^{(l)}\left(h_u^{(l-1)}\right) \rightarrow \boldsymbol{m_u^{(l)} = MSG_r^{(l)}\left(h_u^{(l-1)}\right)},$$

Example:

A linear layer: $m_u^{(l)} = W_r^{(l)} h_u^{(l-1)}$

Heterogeneous Aggregation

- Observation: Each node could receive multiple types of messages from its neighbors, and multiple neighbors may belong to each message type.

- Idea: We can define a 2-stage message passing:

$$h_v^{(l)} = AGG^{(l)}\left(\left\{m_u^{(l)}, u \in N(v)\right\}\right) \rightarrow \boldsymbol{h_v^{(l)}} = \boldsymbol{AGG_{all}^{(l)}}\left(\boldsymbol{AGG_r^{(l)}}\left(\left\{\boldsymbol{m_u^{(l)}}, \boldsymbol{u} \in \boldsymbol{N_r(v)}\right\}\right)\right)$$

Aggregate the messages that belongs to the relation type r with $AGG_r^{(l)}$

Aggregate across edge types with $AGG_{all}^{(l)}$

Example: $\boldsymbol{h_v^{(l)}} = \boldsymbol{Concat}\left(\boldsymbol{Sum}\left(\left\{\boldsymbol{m_u^{(l)}}, \boldsymbol{u} \in \boldsymbol{N_r(v)}\right\}\right)\right)$

Heterogeneous Graph Layers

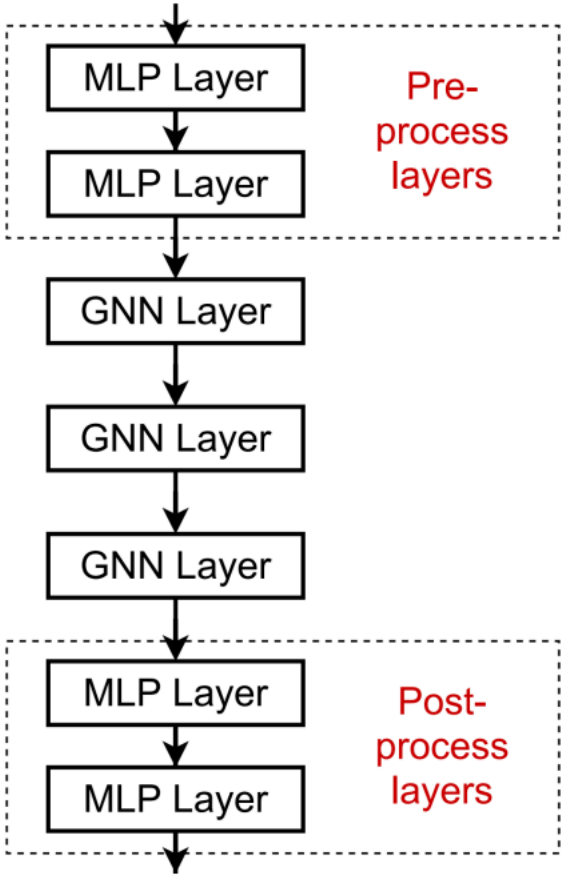
- Heterogeneous pre/post-process layers:
- MLP layers with respect to each node type:

$$h_v^{(l)} = MLP_{T(v)}\left(h_v^{(l)}\right)$$

$T(v)$ is the type of node v

- Other successful GNN designs are also encouraged

for heterogeneous GNNs: skip connections, batch/layer normalization, …



Heterogeneous Graph Manipulation



MACQUARIE University

Graph Structure Manipulation

Recap:

- The graph is **too sparse** → **Add virtual nodes / edges**
- The graph is **too dense** → **Sample neighbors when doing message passing**
- The graph is **too large** → **Sample subgraphs to compute embeddings**

Neighbor and subgraph sampling are also common for heterogeneous graphs.

2 Common options:

sampling **within each relation type** (ensure neighbors from each type are covered), or **sample across the full graph**

Heterogeneous Prediction Heads

MACQUARIE University

Recap:

Node-level prediction:

- $\hat{\boldsymbol{y}}_v = \text{Head}_{\text{node}}(\mathbf{h}_v^{(L)}) = \mathbf{W}^{(H)}\mathbf{h}_v^{(L)}$

Edge-level prediction:

- $\hat{\boldsymbol{y}}_{uv} = \text{Head}_{\text{edge}}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)}) =$
  $\text{Linear}(\text{Concat}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)}))$

Graph-level prediction:

- $\hat{\boldsymbol{y}}_G = \text{Head}_{\text{graph}}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$

Heterogeneous Prediction Heads

MACQUARIE University

Node-level prediction:

- $\hat{\boldsymbol{y}}_v = \text{Head}_{\text{node},\, T(v)}(\mathbf{h}_v^{(L)}) = \mathbf{W}_{T(v)}^{(H)} \mathbf{h}_v^{(L)}$

Edge-level prediction:

- $\hat{\boldsymbol{y}}_{uv} = \text{Head}_{\text{edge},\, r}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)}) = \text{Linear}_r(\text{Concat}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)}))$

Graph-level prediction:

- $\hat{\boldsymbol{y}}_G = \text{AGG}(\text{Head}_{\text{graph},\, i}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall T(v) = i\}))$